Borealis

A framework for inverse problems and optimization that searches the inputs of external codes

Design, methods, parallelization and verification

Yusuke Takahashi
Hokkaido University

20 October 2026 (Borealis version 1.7.0)

Abstract Borealis is a Python framework that searches the input parameters of an external computer code (or of a simple function) so that its output comes close to reference data. The optimization methods (particle swarm optimization, PSO; artificial bee colony, ABC; genetic algorithm, GA; Bayesian optimization) and the codes being driven (the trajectory code Tacode, heat conduction, shape optimization with SU2 and preCICE, and others) are kept apart as "adapters", and one YAML configuration file selects the combination. PSO, ABC and GA can share the evaluations among processes with mpi4py, and PSO and GA also optimize several objectives. For external codes that themselves run under MPI, Borealis can also run as a single process and evaluate the candidates concurrently in threads. This report describes what the code computes and how, how it is parallelized, and how close it gets to the answer on problems whose solution is known. Unless stated otherwise, the numbers come from the verification cases (`report/verification/`) and the measurements (`report/data/`) of the repository, and the figures are made from their output.

Contents

1 Scope of this report **1**

2 Problem setting **1**

3 Structure of the code **2**

4 Optimization methods **3**

5 Parallelization **7**

6 Adapters **8**

7 Output **8**

8 Verification **9**

9 An application: atmospheric density from the orbit of the ISS **13**

10 Known issues **13**

1 Scope of this report

Borealis solves problems of the following form. A computer code takes input parameters $\mathbf{x}$ and returns an output. Reference data, from observation or from another computation, are given, and the output should come close to them. The difference between the output and the reference data is taken as the objective function $f(\mathbf{x})$, and Borealis searches for the $\mathbf{x}$ that makes it small.

It does not look inside the code; it only rewrites the input, runs the code and reads the output, again and again. This is black-box optimization.

This report describes what the code assumes (Section 2), how it is put together (Section 3), what the four methods compute (Section 4), how they are parallelized (Section 5), and how far the answers are from the exact solutions (Section 8). The equations of the methods are written **as they are in the source**, not in the general form of the literature; where they differ from the standard form, this is said where it happens.

`README.md` is the overview. Every key of the configuration file is listed in `report/borealis_config_ja.pdf` (in Japanese). What is not fixed yet or not checked yet is numbered in a development record kept outside the repository, and the text refers to it by number, as in "known issue 7".

2 Problem setting

2.1 Parameters and bounds

The parameters are written in `parameter_optimized.boundary` of the configuration file, as names (`name`) holding components (`component`). Each component has a name (`type`) and a lower and an upper bound (`bound_min`, `bound_max`); all the components in order make the parameter vector $\mathbf{x} = (x_1, \ldots, x_d)$. The `name` is the key used to find the variable to rewrite in the control file of the external code (how it is used differs between adapters).

The bounds act as follows.

- PSO, ABC and GA draw their initial values, and Bayesian optimization its initial points, uniformly within the bounds. Bayesian optimization also searches the maximum of the acquisition within the bounds
- ABC and GA keep new candidates within the bounds
- **PSO does not clip**. Particles can leave the bounds, and the external code then receives values outside them (known issue 21)

2.2 Objective function and penalty

The objective function is the real number (a list of real numbers for several objectives) returned by `objective_function` of the adapter. In adapters that drive an external code it is usually the sum of the squares of the differences between the output and the reference data, or the square root of their mean.

With `flag_penalty: True`, the adapter adds `penalty_value` when the parameters are outside the bounds.

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + P(\mathbf{x}), \qquad P(\mathbf{x}) = \begin{cases} p & \text{if } x_i \notin [x_i^{\min}, x_i^{\max}] \text{ for some } i \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

p is a constant that depends neither on how many parameters are outside nor on how far (known issue 13). Outside the bounds the function is flat, so the penalty alone gives no gradient that brings the particles back.

2.3 Minimization and maximization

All four methods minimize. With `maximize: True`, PSO, ABC and GA pass the sign $s = -1$ to the objective function as an argument, and **the adapter applies the sign** (the penalty is added after the sign). Adapters that apply it declare `flag_apply_sign`; they are now `shapeoptimizer` and `simple_function`. The other adapters return an error, for which maximization has no meaning, and `maximize: True` stops the run at start-up (up to 1.6 they silently kept minimizing;

known issue 1). Bayesian optimization handles the sign itself, so it can maximize with any adapter.

2.4 Failed evaluations

When the external code fails (a non-zero exit status, no result file, or no point that can be compared with the reference data), the adapter returns infinity. PSO, ABC and GA replace NaN or infinite values of the objective by `parameter_optimized.value_nonfinite` (default 10^{30}) with a warning. Up to 1.6 a NaN took over the leader through `np.argmin`, and the error could be computed from old results of a failed external code (known issues 2 and 7).

3 Structure of the code

3.1 Three layers

- `src/borealis.py`: the entry point. It reads the words `adapter.kind_adapter` and `optimizer.kind_optimizer` of the configuration file and loads and combines the corresponding classes
- `src/optimizer_*/`, `src/Bayesian_optimization/`: the optimization methods, which call `objective_function` of the adapter
- `src/adapter_*/`: the side that runs a code and returns the value of the objective function. `src/adapter_template/` has two templates (a minimal one with an analytic function only, and a complete one that drives an external code)

Adapters and optimizers both inherit from the common base class `orbital` (`src/orbital/orbital.py`), which holds directory handling, YAML reading, the penalty and the conversion of time units. MPI is a thin wrapper in `src/mpi/mpi.py`; with one process the code behaves serially.

3.2 Order of calls

`borealis.py` calls the adapter and the optimizer in this order.

1. `adapter.initial_settings(config)`: makes the working directory and copies the template case. **The working directory is removed with its contents first**
2. `adapter.reference_data_setting(config)`: reads the reference data
3. `optimizer.initial_setting(config)`: makes the output directory (also removed first)
4. `optimizer.boundary_setting(config)`: converts the bounds to the form of the method (a list of $(x^{\min}, x^{\max})$ pairs for PSO, ABC and GA, a dictionary from serial names to bounds for Bayesian optimization)
5. `optimizer.drive_optimization(config, adapter.objective_function, ...)`: runs the optimization and writes the results

The objective function is called as `objective_function(x, id, step, sign)`. `x` is an array of shape $(1, d)$ (following the calling convention of GPyOpt, the former library of Bayesian optimization). All four methods pass a unique number `id` for each evaluation, the step `step` and the sign `sign`; the adapter uses `id` as the number of the case directory.

3.3 Driving an external code

`objective_function` of an adapter that drives an external code does roughly the following.

1. Copy the template case to `<work_dir>/<case_dir><number>`
2. Find the variables named by the parameters in the control file and rewrite their values
3. Start the external code and wait for it to end (`subprocess.call`)

4. Read the result file, compare it with the reference data and compute the error, adding the penalty outside the bounds

Every case directory is kept, so the case of the best solution can be taken out and examined afterwards (`src_helper/pickup_optimal_result`). Starting the external code is gathered in `orbital.run_external_code`, which removes a result file copied from the template before the start and checks the exit status and the result file afterwards. The child process does not receive the MPI environment variables (Section 5.2).

4 Optimization methods

Below, N is the number of candidates (particles, employed bees), d the dimension, n the step and $\mathcal{U}(a, b)$ a uniform random number. The random numbers come from the global generator of numpy; with `optimizer.seed`, each rank is seeded with *seed* + *rank*.

4.1 Particle swarm optimization (PSO)

`src/optimizer_pso/optimizer_pso.py`. PSO with an inertia weight [1, 2], with an archive of non-dominated solutions for several objectives [3].

Initialization. The position $\mathbf{x}_i$ of particle i is drawn uniformly within the bounds (with `boundary_initial`, that single range is used for every dimension). The velocity $\mathbf{v}_i$ is drawn from `velocity_initial` if given, and otherwise from $\mathcal{U}(-1, 1)$ **whatever the scale of the parameters**. The personal best value $f(\mathbf{p}_i)$ starts at $+\infty$.

One step. Each step does the following.

1. Evaluate the objective function at the current positions $\mathbf{x}_i^n$ of all particles (NaN and infinite values are replaced by `value_nonfinite`)
2. If the new value dominates the personal best (for one objective, if it is strictly smaller), update the personal best $\mathbf{p}_i$
3. Build the archive **from the current positions of the particles of this step only**: the single best point for one objective, the set of mutually non-dominated points for several
4. Choose the leader $\mathbf{g}$ from the archive: the only point for one objective, a uniformly random member for several
5. Update the velocities and positions

$$\mathbf{v}_i^{n+1} = w\,\mathbf{v}_i^n + c_1\,\mathbf{r}_1 \circ (\mathbf{p}_i - \mathbf{x}_i^n) + c_2\,\mathbf{r}_2 \circ (\mathbf{g} - \mathbf{x}_i^n), \tag{2}$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \mathbf{v}_i^{n+1} \tag{3}$$

Here w is `inertia`, c_1 `cognitive_coef`, c_2 `social_coef`, $\mathbf{r}_1, \mathbf{r}_2$ vectors of independent $\mathcal{U}(0, 1)$ components and $\circ$ the componentwise product. Neither the positions are clipped nor the velocities limited.

What differs from the standard PSO is the leader. In the standard PSO the global best is the best point found so far; in Borealis the leader is **the best of the particles of the current step**, and good points of the past are lost at the next step (a deliberate choice according to the comment in the source; known issue 10). The personal bests remember the past, so the swarm does not forget the best point altogether. A function that chooses the leader of several objectives by crowding distance (`select_global_best_cd`) is written but not used.

Convergence. From the history of the personal best values, the relative change of each particle is

$$r_i^n = \frac{\left\| f(\mathbf{p}_i)^{(n)} - f(\mathbf{p}_i)^{(n-W+1)} \right\|}{\left\| f(\mathbf{p}_i)^{(0)} \right\| + 10^{-15}}, \tag{4}$$

and its mean over the particles, $\bar{r}^n$, is the "swarm residual". W is `windowsize_residual` (default 20) and $f(\mathbf{p}_i)^{(0)}$ the personal best value after the first step. For one objective, the run has converged when $\bar{r}^n \leq$ `tolerance_residual` (default 10^{-5}) and $\bar{r}^n$ differs from the first residual computed (which excludes the apparent convergence of nothing having changed during W steps).

For several objectives, the stagnation of the archive is required as well. With the mean (centroid) $\bar{\mathbf{f}}^n$ of the values of the archive, the archive stagnates when $\|\bar{\mathbf{f}}^n - \bar{\mathbf{f}}^{n-W_a+1}\| \leq$ `tolerance_archive` (default 1.0; W_a is `windowsize_archive`, default 5), and the run has converged when both the residual and the stagnation hold.

Number of evaluations. N per step.

4.2 Artificial bee colony (ABC)

`src/optimizer_abc/optimizer_abc.py`. The ABC of Karaboga [4, 5], for one objective only (that it accepts a setting with several objectives is known issue 12).

N food sources $\mathbf{x}_i$ are drawn uniformly within the bounds and evaluated. Each step has three phases.

Employed bees. For each i, a dimension k and a $j \neq i$ are chosen at random and

$$v_k = x_{ik} + \phi\,(x_{ik} - x_{jk}), \qquad \phi \sim \mathcal{U}(-1, 1) \tag{5}$$

(the other dimensions keep the values of $\mathbf{x}_i$). v_k is clipped to the bounds. If the fitness

$$\mathrm{fit}(f) = \begin{cases} 1/(1+f) & f \geq 0 \\ 1 + |f| & f < 0 \end{cases} \tag{6}$$

improves, $\mathbf{x}_i$ is replaced by $\mathbf{v}$ and its visit counter reset to 0; otherwise the counter is increased by 1.

Onlooker bees. M bees (`num_onlook_bee`) choose food source i with probability $\mathrm{fit}_i / \sum_l \mathrm{fit}_l$ (roulette-wheel selection) and search its neighbourhood in the same way as the employed bees.

Scout bees. A food source whose visit counter **exceeds** `visit_limit` (the old spelling `vist_limit` is still read) is abandoned, drawn again uniformly within the bounds and evaluated. **The best food source is not exempt.**

Best and convergence. ABC keeps the best solution found so far apart from the food sources, so it is not lost when a scout abandons the best source. The output file `output_global.dat` holds the best so far at each step (up to 1.6 it held the best of the food sources at each step, which could get worse; known issue 16, Section 8.1). With $W =$ `windowsize_residual` (default 1), convergence is $\mathrm{mean}_i |f_i^n - f_i^{n-W}| / (|f_i^0| + 10^{-20}) \leq$ `tolerance`, not checked during the first W steps. A step without any improvement makes this 0, so if the run stops too early, increase W (known issue 5).

Number of evaluations. $N + M$ per step, plus the number of scouts. The evaluation number is $10000 \times \mathrm{step} + 1000 \times \mathrm{phase} + \mathrm{bee\ number}$, so the numbers collide when N or M reaches 1000.

4.3 Genetic algorithm (GA)

`src/optimizer_ga/optimizer_ga.py`. A real-coded GA with the survivor selection of NSGA-II [9]. With one objective it is an elitist $(\mu + \lambda)$ GA in which the best individual always survives; with two or more objectives it is NSGA-II itself.

One generation. N children are made from N parents and evaluated, and the next N individuals are chosen from the $2N$ parents and children together. Only the N children are evaluated in a generation; the values of the parents are carried over.

1. **Selection**: `tournament_size` (default 2) individuals are drawn at random, and the one with the lower non-dominated rank, or with the same rank the larger crowding distance, becomes a parent
2. **Crossover**: with probability `crossover_rate` (default 0.9), SBX [7]. For each component with probability 0.5, parents $y_1 < y_2$ give

$$c_{1,2} = \tfrac{1}{2}\left[(y_1 + y_2) \mp \beta_q (y_2 - y_1)\right], \tag{7}$$

   where β_q is drawn from a polynomial distribution with index η_c (`eta_crossover`, default 15) so that the children stay within the bounds. A larger η_c puts the children closer to the parents
3. **Mutation**: each component, with probability `mutation_rate` (default $1/d$), is shifted by polynomial mutation [8], $y \leftarrow y + \delta_q (y^{\max} - y^{\min})$, with δ_q drawn from a distribution with index η_m (`eta_mutation`, default 20) so that it stays within the bounds
4. **Survivor selection**: the $2N$ parents and children are ranked by non-dominated sorting, and N are kept in order of rank and, within a rank, of decreasing crowding distance

Convergence and output. The run stops when the relative change of the mean value of the population over `windowsize_residual` (default 20) generations, $\|\bar{\mathbf{f}}^n - \bar{\mathbf{f}}^{n-W}\|/(\|\bar{\mathbf{f}}^0\| + 10^{-15})$, is at most `tolerance_residual` (default 10^{-5}) and differs from the first value computed (the same guard as PSO). `output_global.dat` holds the first front (the best, for one objective) of every generation; with elitism the best of one objective never gets worse.

Parallelization. Under MPI, selection, crossover and mutation are done by rank 0 alone and sent to all ranks, and only the evaluation of the children is shared (`allgather`), so that all ranks hold the same population even with different random seeds. The concurrent evaluation with `num_parallel` works as in PSO.

Number of evaluations. N for the initial population and N per generation.

The GA up to 1.6 was a prototype that crossed with the mean of the best of the step and shifted all components by the same amount, and it was disabled in `borealis.py`. Version 1.7.0 replaced it.

4.4 Bayesian optimization

`src/Bayesian_optimization/Bayesian_optimization.py`. A Gaussian process is the surrogate model, and the point that maximizes an acquisition function is evaluated next [6]. The Gaussian process and the acquisition function are left to bayesian-optimization [10]; Borealis receives the candidates one at a time, evaluates them and registers the values (suggest and register).

1. The first `initial_design_numdata` points are drawn uniformly within the bounds and evaluated

2. Then, `num_optiter` times, the acquisition function is built from a Gaussian process fitted to all points so far, and the point that maximizes it is evaluated

The acquisition function is the expected improvement (EI), the probability of improvement (PI) or the confidence bound (LCB; the lower bound when minimizing). The exploration parameter ξ of EI and PI is `acquisition_xi`, and the weight κ of LCB is `acquisition_weight`. bayesian-optimization maximizes, so for minimization the values are registered with their sign reversed. As in PSO and ABC, the evaluation number is passed to the objective function, NaN and infinite values are replaced by `value_nonfinite`, and the random seed is `optimizer.seed`. The surrogate is refitted after every evaluation, so the method suits external codes that are expensive per run, with tens of evaluations (Section 8.6). It is not shared among MPI processes.

Up to 1.6 it used GPyOpt [11], which is no longer maintained and does not run with numpy 2, so it was replaced (known issue 22). The settings keep their names; the options that only GPyOpt had (models other than the Gaussian process, the MCMC acquisitions, the plot of the acquisition) are gone.

5 Parallelization

5.1 Sharing the candidates with MPI

Started as `mpirun -n P python3 borealis.py`, PSO, ABC and GA divide the candidates among the ranks.

- **PSO**: the particles are cut evenly into contiguous blocks with `np.array_split` (up to 1.6 the remainder went to the last rank; known issue 9). Each rank evaluates its own particles; at the end of the step the positions, values and numbers are gathered to rank 0 (`gather`), which builds the archive and the leader and sends them to all ranks (`bcast`). Each rank updates the velocities of its own particles. The histories are combined with `allreduce` at the end
- **ABC**: the employed bees are divided evenly with `np.array_split`, and the food sources of all ranks are made equal with `Allreduce` at the end of each phase. The onlookers choose among all food sources and can update one outside their own block, so their updates are collected with `allgather` and applied by all ranks in the same order (when only the own block was collected, updates outside it were lost; known issue 25)
- **GA**: the children are divided with `np.array_split`, and their values are collected with `allgather`. The genetic operations are done by rank 0 and sent with `bcast` (Section 4.3)

Only rank 0 makes directories and writes files. When the external code is serial, this scheme works as it is: each rank is bound to one core, the external code it starts inherits that core, and P external codes run side by side on P cores.

5.2 When the external code runs under MPI: concurrent evaluation

When the external code itself runs under MPI (now the SU2 of `shapeoptimizer`), the scheme above cannot be used.

- Starting the `mpirun` of an external code from a rank started by MPI fails: the child inherits the environment variables of the parent job (`OMPI_*`, `PMIX_*`) and takes itself for one rank of the existing job (known issue 18)
- Even if it starts, the child inherits the binding of the parent rank to one core, and all its MPI processes are packed onto that core
- **Even without mpirun**, importing mpi4py (a singleton `MPI_Init`) adds these variables to the environment on the C side. With `env=None`, `subprocess` passes them to the child,

whose `mpirun` ends with exit code 1 and no message. Python's `os.environ` is the copy taken at start-up and does not contain them, so passing `env=dict(os.environ)` explicitly lets it start (known issue 19; both cases checked with Open MPI 4.1.6)

Borealis is therefore started as a single process without MPI, and PSO calls the evaluations of the particles of each step concurrently in threads (`PSO.num_parallel`). Each thread only waits for a child process, so the global interpreter lock of Python does not matter. The results come back in the order of the particles, and the personal bests and the archive are updated in the same order as in a serial run; the PSO algorithm is unchanged.

On the adapter side, each evaluation takes a "slot" (a set of cores), copies the template and starts the external code with `subprocess.Popen`. The cores of a slot are taken in order from the cores this process may use (`os.sched_getaffinity`; under a job scheduler, the cores given to the job). The cores are passed to the run script in the environment variable `BOREALIS_CPU_SET`, which the script gives to `mpirun -cpu-set`. Several `mpirun` started independently do not know of each other and, unless told, all bind to the first cores, so this setting is needed. A slot is returned only after the external code has ended. If the code ends without writing its result file, `penalty_failure` is returned as the value of the objective and the optimization goes on. Because it is called from several threads at once, the adapter must not use `os.chdir` (which acts on the whole process) and must not keep per-evaluation state in the instance. Only adapters that meet these conditions declare `flag_parallel_evaluation = True`; with any other adapter `num_parallel > 1` stops the run. PSO, GA and ABC support it among the optimizers, and `shapeoptimizer` and `simple_function` among the adapters (known issue 17). In ABC the candidates of the employed bees are made from the food sources at the start of the phase (serially, a later bee refers to a food source updated earlier), so a concurrent run gives a different answer from a serial one; the initialization, onlooker and scout phases, whose candidates do not depend on each other, give the serial answer.

The measured speed is in Section 8.5.

6 Adapters

`kind_adapter`	contents
`simple_function`	analytic objective (shifted sphere functions $f_k(\mathbf{x}) = \lvert\mathbf{x} - 2(k-1)\mathbf{1}\rvert^2$); no external code. Used for verification
`example_externalcode`	template that drives an external code; curve fitting
`tacode`	trajectory code Tacode (Python and Fortran versions); estimates the atmospheric density from orbit observations [12, 13]
`heatcond`	heat conduction; estimates the heat flux from the history of the surface temperature
`shapeoptimizer`	SU2 with shape deformation coupled through preCICE (`src_interface/`); two objectives, drag and moment
`cage`	the cage analysis
`user`	slot for a user-defined adapter

Table 1: Adapters. Their configuration keys are in `report/borealis_config_ja.pdf`.

A new adapter is written by copying a template, implementing the four methods (`__init__`, `initial_settings`, `reference_data_setting`, `objective_function`) and adding its name to the dispatch in `borealis.py`.

A caution for adapters that compare in time. `tacode` looks, for each time of the reference data, for the corresponding point in the computed times and compares by linear interpolation

between the two neighbouring points. The `day` of the time unit `reference.timeunit` is the **sidereal day** (86164.0989 s). Up to 1.6 the search had a structure that, when a reference time lay outside the computed range, compared with the last point of the array through Python's negative index (known issue 6). In the ISS example this happened because the reference was made with calendar days, and the objective kept returning plausible values of 0.05–0.06 while it was comparing two points that were not at the same time (known issue 23). In 1.7.0 points outside the range are not compared, and their number is reported as a warning. **A small objective is no evidence of a correct answer.** Compare the time series of the best solution with the reference data in a figure.

7 Output

The results are written to `<method>.result_dir` as Tecplot ASCII.

- `filename_output` (PSO, ABC, GA): the position (and for PSO the velocity, for GA the non-dominated rank), number, error and residual of every candidate at every step (generation), one Tecplot zone per step
- `filename_global` (PSO, ABC, GA): the number, value and position of the archive (PSO), the best so far (ABC) or the first front (GA; the best for one objective) at every step. For several objectives with PSO, the rows of the last step are the final archive, **not the Pareto front over the whole history** (known issue 10)
- `filename_output` (Bayesian optimization): the points, values and numbers in the order of evaluation

In PSO the number is $nN + i + 1$ (n and i counted from 0) and matches the number of the case directory of the adapter. The tools of `src_helper/` (taking out the best solution, deleting the cases that are not optimal, an animation of the history) find the cases by this number.

8 Verification

The verification uses `simple_function`, whose solution is known, in five dimensions with each component in $[-10, 10]$. The cases are in `report/verification/`, with the random seed fixed by `optimizer.seed`, and `report/run_verification.py` runs them all. Each case takes about a second.

8.1 One objective: the sphere function

$f(\mathbf{x}) = |\mathbf{x}|^2$, with the minimum $f = 0$ at $\mathbf{x} = \mathbf{0}$. PSO has 20 particles with $w = 0.5$ and $c_1 = c_2 = 1.0$; ABC has 20 employed and 20 onlooker bees with `visit_limit` = 10; GA has 20 individuals with the default crossover and mutation.

Figure 1 shows the results.

- PSO met its convergence criterion after 58 steps (1160 evaluations) and stopped at $f = 7.3 \times 10^{-12}$. The best of each step fell almost monotonically and nearly coincides with the best so far; on this problem, a leader that is the best of the current step does not hinder convergence
- ABC ran all 100 steps (about 4000 evaluations), and its best so far was $f = 1.7 \times 10^{-8}$. The best food source of each step (dotted) gets worse by more than an order of magnitude at steps 10 and 27: a scout abandoned the best food source, which had stopped improving, and drew it again. The best so far (solid, `output_global.dat`) falls monotonically (up to 1.6 the file held the dotted values; known issue 16)

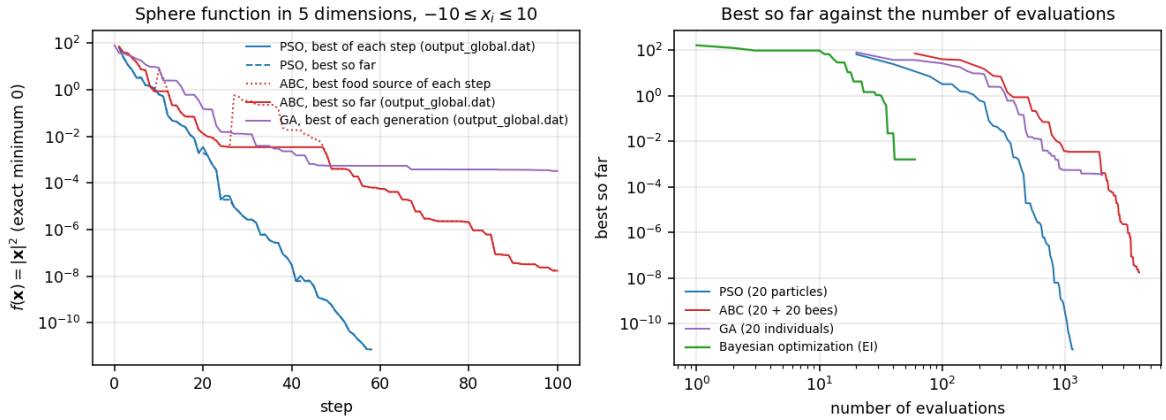


Figure 1: Convergence on the sphere function. Left: against the step. For PSO, the solid line is the archive of each step (the best of that step) and the dashed line the best so far. For ABC, the solid line is `output_global.dat` (the best so far) and the dotted line the best food source of each step (`output_ABC.dat`). For GA, the best of each generation (equal to the best so far, with elitism). Right: the best so far against the number of evaluations, with Bayesian optimization (the evaluations of the scouts of ABC are not counted).

- GA reached $f = 3.2 \times 10^{-4}$ after 100 generations (2020 evaluations). Its best falls monotonically but hardly moves after generation 50 (5.5×10^{-4}): a real-coded GA closes in on the optimum more slowly than PSO. For accuracy, increase `eta_mutation` and `eta_crossover` (children closer to the parents) or use PSO

8.2 Two objectives: the Pareto front

Let $f_1 = |\mathbf{x}|^2$ and $f_2 = |\mathbf{x} - \mathbf{c}|^2$ with $\mathbf{c} = 2 \cdot \mathbf{1}$. By the triangle inequality, for any $\mathbf{x}$

$$\sqrt{f_1} + \sqrt{f_2} = |\mathbf{x}| + |\mathbf{x} - \mathbf{c}| \;\ge\; |\mathbf{c}| = 2\sqrt{5}, \tag{8}$$

with equality only when $\mathbf{x}$ lies on the segment joining $\mathbf{0}$ and $\mathbf{c}$. That segment is the Pareto set, and the Pareto front is $\sqrt{f_1} + \sqrt{f_2} = 2\sqrt{5}$. The distance of an archive point from it is therefore measured by $\delta = |\mathbf{x}| + |\mathbf{x} - \mathbf{c}| - 2\sqrt{5}$ (≥ 0). PSO with 40 particles ran 100 steps (the convergence thresholds were made small enough).

Figure 2 shows the results. The final archive has 31 points, with a mean δ of 0.050 and a maximum of 0.30. The mean of δ falls by two orders of magnitude, from 11.7 at the first step to 0.13 at step 20, and then fluctuates between 0.04 and 0.20 without falling further. We take the reasons for this plateau to be that the archive is built from the particles of the current step only, so better points of earlier steps are not kept, and that the leader is chosen uniformly at random, so the swarm gathers on part of the front (f_1 between 1.8 and 8.6 in the final archive). To obtain the whole front in detail, choose the non-dominated points again from `output_PSO.dat` over all steps (known issue 10).

Solved by GA (40 individuals, 100 generations, survivor selection of NSGA-II), all 40 points of the last generation are in the first front and cover f_1 from 0.05 to 20.0, **the whole front up to both ends** (squares in Figure 2). The mean δ is 0.084 and the maximum 0.21: the mean is slightly larger than with PSO, the maximum smaller. The mean δ keeps falling to the end, 0.29 at generation 20 and 0.10 at generation 60, with no plateau as in PSO. GA keeps parents and children together, so good points survive across generations, and selects by crowding distance, so the points spread over the front. For the whole front with several objectives, and in particular for its ends, use GA.

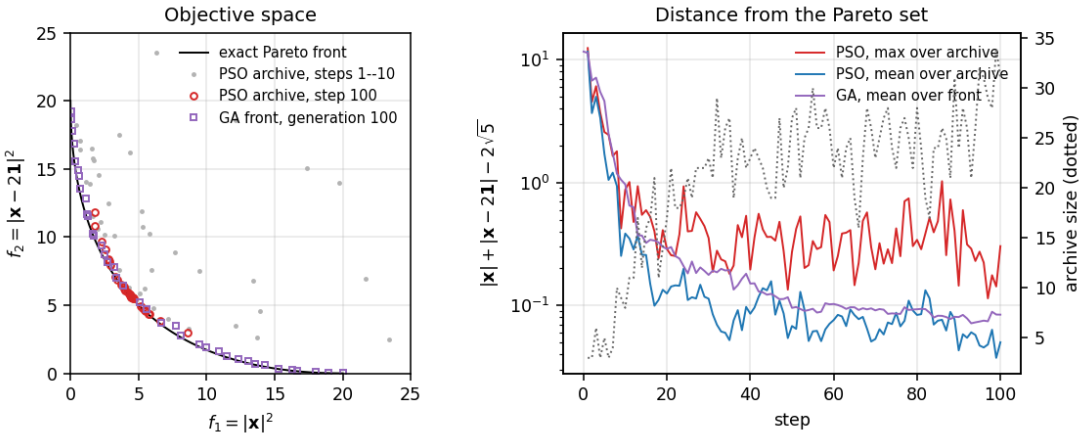


Figure 2: PSO and GA with two objectives. Left: in objective space, the archive of PSO, the first front of the last generation of GA and the exact Pareto front. Right: the distance δ from the Pareto set (maximum and mean over the archive for PSO, mean over the front for GA) and the size of the archive of PSO (dotted).

8.3 Reproducibility

Run twice with the same seed, the `output_global.dat` of the five cases of PSO, ABC and GA agreed bit for bit. In a serial run the answer of Borealis is fixed by the seed alone. With the concurrent evaluation of PSO and GA (`num_parallel`) the random numbers are drawn outside the threads, so the answer is the same as in a serial run. The concurrent evaluation of ABC changes the answer in the employed phase only, and does not depend on the number of threads (both checked by tests). Under MPI each rank has its own random stream, and the answer changes with the number of processes.

8.4 Tests and CI

`tests/` holds pytest tests, run by `python3 -m pytest tests` (42 tests, about 27 s on the development machine; the 3 of Bayesian optimization are skipped where bayesian-optimization is not installed):

- the tools of `orbital`: time units, the default path, the search for the corresponding point (checked against the former loop, and outside the range), the geodetic round trip, the check of objective values, and starting an external code (exit status, an old result file, and no MPI environment variables leaking to the child)
- the configuration check: warnings for unknown and obsolete keys, and no unknown key in any `borealis.yml` of the repository
- the methods end to end: convergence and reproducibility of PSO, the concurrent evaluation giving the serial answer, the sign of maximization, approach to the Pareto set with two objectives, a monotone best so far of ABC, ABC refusing several objectives, GA keeping crossover and mutation within the bounds, its non-dominated sorting, monotone best, reproducibility, concurrent evaluation and covering of the front with two objectives, and convergence, reproducibility, maximization and the refusal of unavailable models in Bayesian optimization
- MPI: PSO, ABC and GA with 13 candidates on 3 processes (only where `mpirun` exists)
- `shapeoptimizer` with a dummy external code without MPI: concurrent evaluation, one and two objectives, the core assignment, and the penalty on failure

On the sources of version 1.6, 16 of 18 of these tests fail, which confirms that the tests catch

the bugs that were fixed. GitHub Actions (`.github/workflows/ci.yml`) runs the syntax check, the tests and the verification cases of this report with Python 3.9 and 3.12 on every push and pull request.

8.5 Speed of the concurrent evaluation

The concurrent evaluation of Section 5.2 was measured on an AMD Ryzen 9 9900X (12 physical cores, 24 logical) with Open MPI 4.1.6. Two external codes were used. One is a dummy MPI program that is started by `mpirun -np 2 -cpu-set`, sleeps for 2 s and writes the core binding of each rank (`sched_getaffinity`). The other is the SU2 + preCICE case `testcase/work_shapeoptimizer/work/work_cylinder2D_multiobj`, shortened to 6 particles and 2 steps. Both used `num_parallel` = 3 and 2 cores per case.

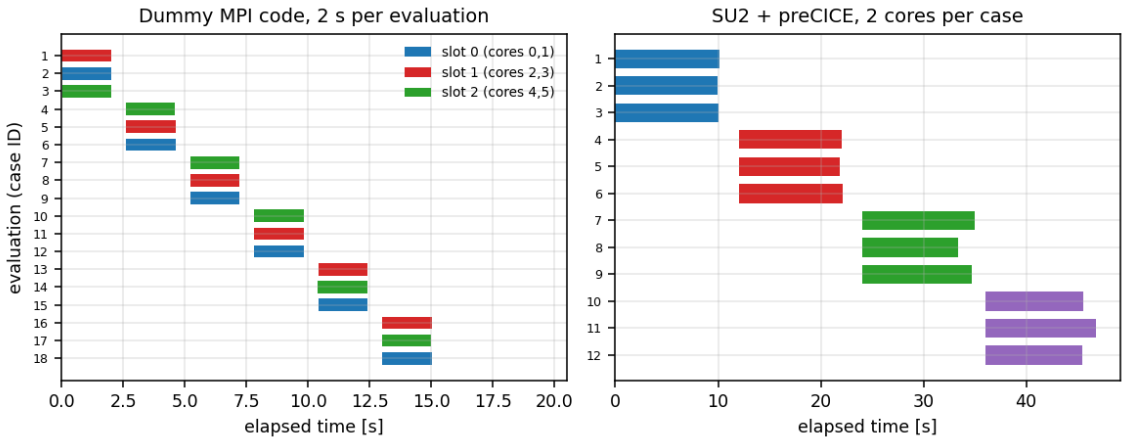


Figure 3: Timeline of the concurrent evaluation. Left: the dummy MPI program (colour = slot). Right: SU2 + preCICE (colour = group within a step).

external code	evaluations	one [s]	total [s]	one by one [s]	speed-up
dummy MPI program	18	2.00	15.0	36.0	2.4
SU2 + preCICE	12	9.3–10.9	46.8	120.7	2.6

Table 2: Speed of the concurrent evaluation (`num_parallel` = 3). "One by one" is the sum of the times of the evaluations.

Figure 3 and Table 2 show the results.

- Three cases ran at a time, and the ranks of each case were bound to the cores assigned to it (0,1 / 2,3 / 4,5, with their SMT siblings), with no overlap
- The speed-up stays below the ideal 3 because of the gaps between the groups: 0.6 s for the dummy (the interval `polling_interval` at which the result file is looked for was 0.2 s) and about 2 s for SU2 (the default of 3 s). PSO waits for all particles of a step, so the slowest case of a group sets its length. **When one evaluation is short, shorten** `polling_interval`
- With the dummy made to fail always, all 6 evaluations returned `penalty_failure` and the optimization ran to its end without stopping

It has not been checked yet in the job environment of a supercomputer (known issue 20). If the cores given to the job do not start at number 0, the absolute numbers of `-cpu-set` miss them.

8.6 Bayesian optimization

The same five-dimensional sphere function was solved by Bayesian optimization (EI, $\xi = 0.01$, 10 initial points and 50 iterations; `report/verification/bo_sphere`). The right of Figure 1 shows the best so far against the number of evaluations, next to PSO and ABC.

- Bayesian optimization reached $f = 1.6 \times 10^{-3}$ at the 42nd evaluation and did not improve in the remaining 18
- To reach the same value, PSO needed 420 evaluations (21 steps), GA 840 (41 generations) and ABC about 1940 (48 steps)
- Beyond that accuracy (below 10^{-6}), Bayesian optimization does not get within 60 evaluations. Fitting the Gaussian process and maximizing the acquisition took 12 s in all for the 60 evaluations (development machine)

Bayesian optimization suits external codes that are expensive per run, with tens of evaluations; PSO suits cheap evaluations where accuracy is needed. Run twice with the same seed, all points evaluated and their values agreed.

9 An application: atmospheric density from the orbit of the ISS

`testcase/work_tacode_fortran` has three examples that estimate the atmospheric density at 410–440 km from the orbit of the ISS. The reference orbit is the TLE of Celestrak propagated by SGP4 and reconstructed by Gaussian-process regression from intermittent pseudo GNSS observations. Tacode computes the orbit, and PSO (100 particles, 50 steps) searches for the density correction that makes the difference in altitude small. The three differ in how the correction is given.

case	correction	converged	objective	RMS of altitude
`work_tacode_iss`	factor per altitude band (4 bands, a step function)	–	0.0008	0.06 km
`work_tacode_iss_pchip`	the same control points joined smoothly by PCHIP	57 steps	0.00095	0.071 km
`work_tacode_iss_f107`	the solar radio flux F10.7 optimized as a single quantity, the density rebuilt by NRLMSISE-00	42 steps	0.00089	0.067 km

Table 3: Results of the ISS examples. The numbers are taken from the record of the runs of August 2026 (`CHANGELOG.md`); they were not recomputed for this report.

All three fit the orbit about equally well, and the F10.7 approach does so with a single physically interpretable quantity (F10.7A $\approx$ 128). The reference orbit, however, comes from SGP4 and not from a measured density, so the correction largely absorbs the difference between the orbit model of Tacode and SGP4. It should not be read directly as the physical atmospheric density.

Two problems that silently corrupted the objective were found along the way: the confusion of sidereal and calendar days of Section 6 (known issue 23), and the atmosphere table of Tacode not covering the altitude of the ISS, so that the density correction had in fact no effect (known issue 24). Both were found not from the value of the objective but by looking at the orbit and the density profile of the best solution in figures.

10 Known issues

The list and the details are in the development record kept outside the repository. Version 1.7.0 implemented GA, replaced the library of Bayesian optimization by bayesian-optimization (22) and fixed the issues that silently corrupted the answer (the sign of maximization, NaN, the factor of `hour`, the wrap-round of the corresponding time, failures of the external code, the default path, `shapeoptimizer` with one objective) and covers them by tests. What remains:

- Unknown keys give a warning but do not stop the run (4)
- The convergence check of ABC can hold at a step without improvement (5)
- The PSO archive and leader are chosen from the particles of the current step only, and neither positions nor velocities are limited (10, 21)
- The penalty is a constant that does not depend on the size of the violation (13)
- The concurrent evaluation is not available in Bayesian optimization and only in some adapters (17); the core assignment on a supercomputer is not checked yet (20)
- The `weight_time` of `tacode` does not enter the error (27)

References

[1] J. Kennedy and R. Eberhart, "Particle swarm optimization," *Proc. ICNN'95*, vol. 4, pp. 1942–1948, 1995.

[2] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," *Proc. IEEE ICEC*, pp. 69–73, 1998.

[3] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga, "Handling multiple objectives with particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 256–279, 2004.

[4] D. Karaboga, "An idea based on honey bee swarm for numerical optimization," Technical Report TR06, Erciyes University, 2005.

[5] D. Karaboga and B. Basturk, "A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm," *J. Global Optim.*, vol. 39, pp. 459–471, 2007.

[6] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *J. Global Optim.*, vol. 13, pp. 455–492, 1998.

[7] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," *Complex Systems*, vol. 9, pp. 115–148, 1995.

[8] K. Deb and M. Goyal, "A combined genetic adaptive search (GeneAS) for engineering design," *Computer Science and Informatics*, vol. 26, no. 4, pp. 30–45, 1996.

[9] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, 2002.

[10] F. Nogueira and contributors, "Bayesian Optimization: Open source constrained global optimization tool for Python," `https://github.com/bayesian-optimization/BayesianOptimization`, version 3.4.0, 2026.

[11] The GPyOpt authors, "GPyOpt: A Bayesian optimization framework in Python," `http://github.com/SheffieldML/GPyOpt`, 2016.

[12] Y. Takahashi, M. Saito, N. Oshima, and K. Yamada, "Trajectory reconstruction for nanosatellite in very low Earth orbit using machine learning," *Acta Astronautica*, vol. 194, pp. 301–308, 2022.

[13] T. Sakai and Y. Takahashi, "Atmospheric density estimation in very low Earth orbit based on nanosatellite measurement data using machine learning," *Aerospace Science and Technology*, vol. 153, 109418, 2024.